DoomScript & Gameplay Code API Documentation

Part 4: DoomScript Language Reference

Types · Operators · Control Flow · Objects & Inheritance
Threading Model · Entity References · Preprocessor

SE2 Engine — Based on Doom 3 BFG Edition
neo/d3xp/ Codebase

Table of Contents

1. Language Overview

DoomScript is a custom scripting language embedded in the idTech 4 / SE2 engine. It provides C-like syntax for defining gameplay logic, AI behavior, weapon state machines, and map scripting. DoomScript compiles to bytecode that runs on a cooperative multithreading virtual machine (the idInterpreter), with each script thread executing until it voluntarily yields via wait(), waitFrame(), or similar calls.

DoomScript is not a general-purpose language. It is specifically designed for gameplay scripting and has these defining characteristics:

It uses a fixed set of primitive types (float, vector, string, entity, boolean) with no arrays, maps, or dynamic allocation. It supports single-inheritance object-oriented programming via the **object** keyword, with virtual method dispatch. It connects to C++ through **scriptEvent** declarations, which bind script-callable functions to C++ event handlers. It features cooperative multithreading where threads yield explicitly rather than being preempted. And it uses a **$-prefixed entity reference** syntax for accessing named entities directly.

1.1 Compilation Pipeline

The compilation pipeline has three stages. The source .script and .def files are loaded by the idProgram object. The idCompiler (Script_Compiler.cpp) performs single-pass compilation, parsing tokens and emitting bytecode statements directly—there is no separate AST stage. The resulting bytecode statements are stored in the idProgram's statement list and executed by the idInterpreter (Script_Interpreter.cpp), one instruction at a time per thread.

1.2 Source File Organization

Scripts are loaded in a specific order controlled by the engine. The base script definitions file (doom_defs.script) is loaded first, establishing all #define constants, followed by event declarations (doom_events.script) that declare scriptEvent prototypes, then base class definitions (ai_base.script, weapon_base.script, etc.), and finally entity-specific scripts. Map-specific scripts (mapname.script) are compiled when a level loads. All scripts share a single global program namespace managed by idProgram.

2. Data Types

DoomScript has a small, fixed set of data types. There are no user-defined value types, no arrays, no pointers accessible from script, and no dynamic allocation. All types are defined as idTypeDef objects in Script_Program.cpp.

2.1 Primitive Types

Type	Internal Size	Description
`float`	sizeof(intptr_t)	The primary numeric type. Used for all integers and floating-point values. There is no separate int type in script—DoomScript uses float for everything numeric, including loop counters, boolean-like checks, bitfield constants, and entity numbers returned from sys calls.
`vector`	E_EVENT_SIZEOF_VEC (3 floats)	A 3-component vector (x, y, z). Used for positions, directions, angles, and colors. Written as literal with single-quoted components: '1 0 0' for a unit X vector. Components accessed with _x, _y, _z suffixes.
`string`	MAX_STRING_LEN (128 bytes)	Fixed-length string buffer. Supports concatenation with + operator. Can auto-convert from float, boolean, and vector via implicit store opcodes (STORE_FTOS, STORE_BTOS, STORE_VTOS).
`entity`	sizeof(intptr_t)	A reference to a game entity. Stored internally as an entity number (index + 1, where 0 = null). Can be compared with == and !=. Use $entityname syntax for compile-time named references.
`boolean`	sizeof(int)	A true/false value. Interconvertible with float (0 = false, nonzero = true). Used extensively for AI state flags (AI_DEAD, AI_PAIN, etc.) and weapon status flags (WEAPON_ATTACK, WEAPON_RELOAD).
`void`	0	Used only as a function return type indicating no return value.

2.2 Object Type

The **object** type (ev_object) is a reference type for script-defined objects. Internally it is stored the same way as entity (an entity number pointer), because every script object is always attached to an entity. The object type system supports single inheritance and virtual function dispatch. See Section 5 for full object system documentation.

2.3 Special Internal Types

These types exist in the compiler/interpreter but are not directly usable in script code:

Type	Purpose
`scriptevent`	Marks a function prototype as an event binding (the scriptEvent keyword).
`namespace`	Used for the namespace keyword to group definitions.
`field`	Internal type for object member field access via the dot operator.
`function`	Internal type representing a callable function.

`virtualfunction`	Internal type for virtual (overridable) object method dispatch.
`pointer`	Internal type used during assignment to object fields.
`jumpoffset`	Internal type used only by jump/branch opcodes.
`argsize`	Internal type used for function call and thread argument size tracking.

2.4 Type Conversions

DoomScript supports several implicit conversions that the compiler inserts automatically. These are handled through dedicated store/push opcodes:

From	To	Opcode	Behavior
`float`	`string`	`STORE_FTOS`	Converts float to its string representation
`boolean`	`string`	`STORE_BTOS`	Converts to "1" or "0"
`vector`	`string`	`STORE_VTOS`	Converts to "x y z" format
`float`	`boolean`	`STORE_FTOBOOL`	0.0 becomes false, nonzero becomes true
`boolean`	`float`	`STORE_BOOLTOF`	false becomes 0.0, true becomes 1.0
`entity`	`object`	`STORE_OBJENT`	Cast entity ref to a typed object (runtime type check)
`object`	`entity`	`STORE_ENTOBJ`	Widening cast: always succeeds

String concatenation via the + operator also triggers implicit conversion: float + string, string + float, vector + string, and string + vector all produce string results via dedicated ADD_FS, ADD_SF, ADD_VS, ADD_SV opcodes.

2.5 Vector Component Access

Individual components of a vector variable are accessed using underscore suffixes:

```
vector pos;
pos = '100 200 50';
float height = pos_z;        // 50
pos_x = pos_x + 10;         // modify X component
```

This syntax works on any vector-typed expression including function return values and object fields. The _x, _y, _z suffixes are parsed by the compiler as special field access operations.

3. Literals & Constants

3.1 Numeric Literals

Float literals follow standard C syntax: 42, 3.14, 0.5, -1. There is no integer literal type—all numeric literals are floats. The compiler recognizes negative numbers and handles them via the NEG_F opcode when used in expressions.

3.2 String Literals

String literals use double quotes: "hello world". The maximum length is MAX_STRING_LEN (128 bytes). String concatenation uses the + operator and supports auto-conversion from numeric types.

3.3 Vector Literals

Vectors use single-quoted space-separated components:

```
vector origin = '0 0 0';
vector red    = '1 0 0';     // also used for colors
vector dir    = '0 0 -1';    // downward direction
```

3.4 Entity Name References ($)

The **$** prefix creates a compile-time entity reference. When the entity spawns, it links itself to this reference by name:

```
$light_hallway.setColor( '1 0 0' );    // set by entity name
$trigger_door.remove();                 // remove named entity
$player1.setHealth( 100 );              // reference to player
```

If the named entity does not exist at the time the code executes, the reference evaluates to null (entity number 0), which will cause a runtime warning if you try to call methods on it. The $-syntax creates an idVarDef of type entity at compile time, and entity spawning fills it in via idGameLocal::FindEntity or similar lookups.

> ***Note:*** *Entity name references are resolved at entity spawn time, not at compile time. A $name reference is valid only after the referenced entity has spawned.*

3.5 The #define Preprocessor

DoomScript supports C-style #define macros via the built-in idParser (which shares the same preprocessor as the C++ codebase). Supported preprocessor directives include:

Directive	Description
`#define NAME value`	Simple text substitution. Used extensively for constants.
`#define MACRO( a ) expr`	Function-like macro with parameters.
`#ifdef / #ifndef / #else / #endif`	Conditional compilation blocks.
`#include "file.script"`	Not typically used directly; file loading is managed by idProgram. But the parser does support it.

`#undef NAME`	Undefines a previously defined macro.

Common patterns from doom_defs.script:

```
#define TRUE            1
#define FALSE           0
#define NULL            0
#define GAME_FRAMETIME  ( 1.0f / GAME_FPS )
#define BIT( num )      ( 1 << ( num ) )
#define DEG2RAD( a )    ( (a) * (M_PI / 180.0f ) )

// Convenience macros for common patterns:
#define eachFrame       for( 0; 1; waitFrame() )
#define waitUntil( x )  while( !( x ) ) { waitFrame(); }
```

Note: *The eachFrame and waitUntil macros are extremely common in weapon and AI scripts. eachFrame creates an infinite loop that yields every frame. waitUntil(condition) spins waiting for a condition to become true, yielding each frame.*

4. Operators

DoomScript operators are defined in the opcodes[] table in Script_Compiler.cpp. Each operator has an explicit priority level that determines evaluation order. Lower priority numbers bind tighter (evaluated first).

4.1 Operator Precedence Table

From highest to lowest priority:

Priority	Operators	Description	Associativity
1	`++ -- .`	Postfix increment/decrement, member access (dot)	Left
2	`() thread`	Function call, thread launch	Left
3	`* / %`	Multiplication, division, modulo	Left
4	`+ -`	Addition, subtraction (also string concatenation)	Left
5	`== != <= >= < >`	Comparison and equality	Left
5	`!`	Logical NOT (same priority as comparisons)	Right (unary)
5	`~`	Bitwise complement (same priority as comparisons)	Right (unary)
6	`&& ||`	Logical AND, logical OR	Left
6	`& |`	Bitwise AND, bitwise OR	Left
7	`= += -= *= /= %= &= |=`	Assignment and compound assignment	Right

4.2 Arithmetic Operators

Arithmetic operators work on float and vector types with the following combinations:

Operator	Left Type	Right Type	Result Type	Opcode
*	float	float	float	MUL_F
*	vector	vector	float	MUL_V (dot product)
*	float	vector	vector	MUL_FV (scalar * vec)
*	vector	float	vector	MUL_VF (vec * scalar)
/	float	float	float	DIV_F
%	float	float	float	MOD_F
+	float	float	float	ADD_F
+	vector	vector	vector	ADD_V
+	string	string	string	ADD_S (concatenation)
-	float	float	float	SUB_F
-	vector	vector	vector	SUB_V

> ***Note:*** *The * operator between two vectors performs a dot product, not a component-wise multiply. Use sys.CrossProduct() for cross products.*

4.3 Comparison Operators

All comparison operators return a float result (1.0 for true, 0.0 for false). Equality comparisons are available for all major types:

Operator	Supported Types	Notes
`==`	float, vector, string, entity, object	Entity-object and object-entity comparisons are supported via EQ_EO, EQ_OE, EQ_OO opcodes
`!=`	float, vector, string, entity, object	Same cross-type combinations as ==
`<= >= < >`	float only	Ordering comparisons only work on floats

4.4 Logical and Bitwise Operators

Logical operators (&&, ||) work on float and boolean types and support mixed operand types via dedicated opcodes: AND_BOOLF, AND_FBOOL, AND_BOOLBOOL, and their OR equivalents. Bitwise operators (& and |, via BITAND/BITOR opcodes) operate on float values treated as integers. The bitwise complement (~) via COMP_F is a unary operator that also treats the float as an integer.

4.5 Unary Operators

Operator	Operand	Opcode	Description
`++`	float	UINC_F	Increment by 1 (also UINCP_F for object field access)
`--`	float	UDEC_F	Decrement by 1 (also UDECP_F for object field access)
`-`	float / vector	NEG_F / NEG_V	Arithmetic negation
`!`	float / bool / string / entity / vector	NOT_F / NOT_BOOL / NOT_S / NOT_ENT / NOT_V	Logical NOT (tests for zero/null/empty)
`~`	float	COMP_F	Bitwise complement (integer interpretation)

4.6 Compound Assignment Operators

Compound assignment operators (+=, -=, *=, /=, %=, &=, |=) are parsed from the punctuation table and are available for float types. The vector types support +=, -=, *=, and /= through the UADD_V, USUB_V, UMUL_V, UDIV_V opcodes. These operators are right-associative.

5. Objects & Inheritance

DoomScript's object system provides single-inheritance object-oriented programming. Every script object is attached to a game entity—there are no standalone script objects. The object system is the primary way to define behavior for AI, weapons, players, and other gameplay entities.

5.1 Object Declaration

Objects are declared with the **object** keyword at namespace scope (they cannot be nested inside functions or other objects):

```
object weapon_shotgun : weapon_base {
    // member variables (fields)
    float       next_attack;
    float       spread;
    string      customSkin;

    // method prototypes (must be declared here)
    void        init();
    void        destroy();
    void        Raise();
    void        Lower();
    void        Idle();
    void        Fire();
    void        Reload();
};
```

The syntax is: **object** objectname [: parentname] { fields and method prototypes }; If no parent is specified, the object inherits from the built-in root **object** type. All fields and method prototypes must be declared inside the braces. Method bodies are defined separately using the :: scope resolution syntax.

5.2 Method Definition

Method bodies are defined outside the object block using scope resolution:

```
void weapon_shotgun::init() {
    next_attack = 0;
    spread = getFloatKey( "spread" );
    weaponState( "Raise", 0 );
}

void weapon_shotgun::Fire() {
    float ammoClip;
    next_attack = sys.getTime() + SHOTGUN_FIRERATE;
    ammoClip = ammoInClip();
    launchProjectiles( SHOTGUN_NUMPROJECTILES, spread, 0, 1.0, 1.0 );
    playAnim( ANIMCHANNEL_ALL, "fire" );
    waitUntil( animDone( ANIMCHANNEL_ALL, SHOTGUN_FIRE_TO_IDLE ) );
    weaponState( "Idle", SHOTGUN_FIRE_TO_IDLE );
```

```
}
```

Inside a method, you have implicit access to the object's fields and other methods via an implicit **self** pointer. The compiler automatically resolves field names to the correct object using the self def and OP_INDIRECT / OP_UINCP_F opcodes.

5.3 Constructors and Destructors

The special method name **init()** is the constructor. The compiler automatically chains constructors: when init() is defined, the compiler emits a call to the parent class's init() before executing the derived class's constructor body. The special method name **destroy()** is the destructor. Similarly, the compiler chains destructors: it patches all return statements to jump to a call to the parent's destroy() before actually returning.

```
void weapon_shotgun::init() {
    // parent weapon_base::init() is called automatically first
    next_attack = 0;
    spread = getFloatKey( "spread" );
    weaponState( "Raise", 0 );
}

void weapon_shotgun::destroy() {
    // cleanup code here
    // parent weapon_base::destroy() called automatically after
}
```

Note: *Constructor chaining goes up the hierarchy to the base class first (top-down). Destructor chaining goes bottom-up: the derived destructor runs first, then the parent destructor.*

5.4 Inheritance & Virtual Dispatch

All methods in DoomScript are virtual by default. When a method is called on an object reference, the interpreter uses OP_OBJECTCALL which resolves the function through the object's actual type definition (obj->GetTypeDef()->GetFunction()), enabling polymorphic behavior. The idTypeDef::MatchesVirtualFunction() method validates that overrides have compatible signatures, with the special rule that the first parameter (self) may be a derived type.

```
object monster_base : ai {
    float   check_attacks();     // virtual by default
    void    do_attack( float attack_flags );
};

object monster_demon : monster_base {
    float   check_attacks();     // override
    void    do_attack( float attack_flags );  // override
};

// The override is resolved at runtime through the object's vtable
```

5.5 Member Access

Object fields are accessed using the dot operator. The compiler generates OP_INDIRECT_* opcodes (INDIRECT_F, INDIRECT_V, INDIRECT_S, INDIRECT_ENT, INDIRECT_BOOL, INDIRECT_OBJ) for reads, and OP_STOREP_* opcodes for writes (which first compute the ADDRESS and then store through the pointer). Within the object's own methods, fields are accessed directly by name without the dot operator (the implicit self is injected by the compiler).

```
// From outside the object:
entity ent = $monster_demon_1;
// Entity-to-object access uses event calls, not direct field access

// From inside the object's own methods:
void monster_base::state_Combat() {
    run = true;           // direct field access via implicit self
    float dist = enemyRange();  // method call on self
}
```

5.6 The Entity-Object Relationship

Every script object is bound to an entity via the "scriptobject" spawnarg in the entity's entityDef. When an entity spawns, it creates an idScriptObject instance, allocates memory for the object's fields (scriptObject.SetType()), and calls the constructor. The C++ side accesses script object data through idScriptVariable<> template helpers that look up fields by name and type in the script object's data buffer. Script-side C++ linked variables (like WEAPON_ATTACK, AI_DEAD) are bridged via idScriptBool which directly links a C++ bool to a boolean field in the script object's memory.

6. Control Flow

DoomScript supports C-style control flow statements. These are parsed by the respective ParseIfStatement(), ParseWhileStatement(), ParseForStatement(), and ParseDoWhileStatement() methods in idCompiler.

6.1 if / else

```
if ( ammoClip > 0 ) {
    weaponState( "Fire", IDLE_TO_FIRE );
} else if ( ammoAvailable() > 0 ) {
    weaponState( "Reload", IDLE_TO_RELOAD );
} else {
    weaponState( "NoAmmo", IDLE_TO_NOAMMO );
}
```

The condition is evaluated as float or boolean. Zero/false takes the else branch; any nonzero value takes the if branch. Internally uses OP_IFNOT for the conditional jump.

6.2 while

```
while ( 1 ) {
    if ( WEAPON_LOWERWEAPON ) {
        weaponState( "Lower", IDLE_TO_LOWER );
    }
    waitFrame();     // yield to the engine each iteration
}
```

Infinite loops (while(1)) are the standard pattern for weapon idle states and AI behavior loops. The loop must contain a waitFrame() or wait() call to yield, otherwise the interpreter's runaway loop detector (5,000,000 instructions) will trigger an error.

6.3 for

```
for ( i = 0; i < count; i++ ) {
    // loop body
}
```

The for loop compiles to the same structure as a while loop: the initializer runs once, then the condition check and body repeat, with the increment expression executing after each body iteration before re-checking the condition.

6.4 do-while

```
do {
    // execute at least once
    waitFrame();
} while ( !AI_ENEMY_VISIBLE );
```

6.5 break and continue

break exits the innermost loop immediately. **continue** jumps to the next iteration (the increment expression in for loops, or the condition check in while/do-while). These are compiled as

OP_BREAK and OP_CONTINUE placeholder opcodes that are patched by PatchLoop() to the correct jump targets after the loop is fully parsed. Using break or continue outside of a loop causes a compile error.

6.6 return

```
float monster_base::check_attacks() {
    if ( !canSeeEnemy() ) {
        return 0;
    }
    return ATTACK_MISSILE;
}
```

Functions with a return type must return a value. Void functions can use bare return; to exit early. The compiler emits OP_RETURN which pops the call stack and resumes the caller. In destructors, return statements are patched to jump to the parent destructor call before actually returning.

7. Functions

7.1 Free Functions

Functions can be declared at namespace scope (outside any object). They are called directly by name:

```
void main() {
    // map entry point - called when the level loads
    sys.println( "Map loaded" );
    thread spawn_monsters();
}

void spawn_monsters() {
    sys.wait( 3 );
    sys.trigger( $trigger_spawn_wave1 );
}
```

The **main()** function is special: when a map loads, the engine looks for a function named "main" in the compiled program and creates an idThread to execute it (see idWorldspawn::Spawn()). Free functions can also be invoked via "call" key-value pairs on the worldspawn entity.

7.2 Function Prototypes

Inside object declarations, method prototypes must be declared before definitions. For free functions, prototypes are optional—you can provide just the definition. Prototypes look like the declaration without a body:

```
void myFunction( float value, string name );  // prototype

void myFunction( float value, string name ) {  // definition
    sys.println( name + ": " + value );
}
```

7.3 Calling Conventions

DoomScript uses a stack-based calling convention. Arguments are pushed onto the interpreter's local stack using OP_PUSH_* opcodes, then OP_CALL enters the function. The called function accesses parameters as local variables. Return values are stored in the global return variable (gameLocal.program.returnDef). The call stack has a fixed depth limit (MAX_STACK_DEPTH = 64) and the local stack is limited to LOCALSTACK_SIZE (6144 bytes).

There are three distinct call mechanisms depending on what is being called:

Call Type	Opcode	Description
Direct function call	`OP_CALL`	Calls a known function. Used for free functions and known method calls.
Object method call	`OP_OBJECTCALL`	Calls a virtual method on an object. Resolves function through the object's runtime type for polymorphism.
Event call (scriptEvent)	`OP_EVENTCALL`	Calls a C++ event handler through the event system. Used for all scriptEvent-declared functions called on entities.

System call (sys.*)	`OP_SYSCALL`	Calls a system event on the global sys object. Dispatched through idThread's event handling.
Thread launch	`OP_THREAD`	Creates a new idThread to run a free function concurrently.
Object thread launch	`OP_OBJTHREAD`	Creates a new idThread to run an object method concurrently.

7.4 The sys Object

The **sys** object is a special built-in that provides access to system-level functions. These are scriptEvent functions declared on idThread and accessed via the OP_SYSCALL mechanism. In script, they are called with the sys. prefix:

```
float t = sys.getTime();
float r = sys.random( 100 );
sys.println( "Random: " + r );
sys.wait( 1.5 );
sys.trigger( $some_entity );
entity ent = sys.getEntity( "monster_demon_1" );
```

The compiler detects the sys. prefix and generates OP_SYSCALL instead of OP_EVENTCALL. It validates that the function is actually responded to by idThread's type.

7.5 The thread Keyword

The **thread** keyword launches a function call as a new concurrent thread, returning the thread number:

```
float threadNum = thread spawn_monsters();
sys.println( "Started thread: " + threadNum );

// For object methods:
float threadNum = thread $my_entity.doSomething();
```

When the compiler encounters the **thread** keyword, it sets the callthread flag which changes OP_CALL to OP_THREAD (or OP_OBJECTCALL to OP_OBJTHREAD). The thread keyword cannot be used with scriptEvent (built-in) functions—only with script-defined functions. The new thread gets a copy of the arguments and runs independently.

> ***Note:*** *You cannot use thread with sys.* calls or scriptEvent functions. The Error "Cannot call built-in functions as a thread" will be thrown.*

8. Threading Model

DoomScript uses cooperative multithreading. Multiple script threads can exist simultaneously, but only one thread executes at any given time. Threads must explicitly yield control—there is no preemptive switching. This makes scripts deterministic and avoids race conditions.

8.1 Thread Creation

Threads are created in several ways:

Using the **thread** keyword in script: thread myFunction() creates a new idThread that begins executing myFunction. From C++ via idEntity::ConstructScriptObject(): when an entity spawns, it creates a thread to run the script object's constructor. From C++ via idEntity::RunScriptFunc() (an SE2 custom extension): dynamically calls a function on an entity's script object from C++. From map worldspawn: the main() function and any "call" key-value functions are each launched as separate threads.

8.2 Yielding & Waiting

Threads yield control using these mechanisms:

Script Call	Behavior
`waitFrame()`	Suspends thread until the next game frame. This is the most common yield. The thread posts an EV_Thread_Execute event to itself for 1ms in the future.
`wait( float seconds )`	Suspends for the specified duration. Sets waitingUntil = gameLocal.time + seconds * 1000 and posts a delayed EV_Thread_Execute.
`waitFor( entity mover )`	Suspends until the specified mover entity signals completion. Sets waitingFor to the entity number.
`waitForThread( float threadNum )`	Suspends until the specified thread terminates. Sets waitingForThread pointer.

When a thread yields, its idInterpreter sets doneProcessing = true, which causes Execute() to return without destroying the thread. The thread is later resumed when its wait condition is satisfied (via the posted event or a callback).

8.3 Thread Lifecycle

A thread follows this lifecycle: it is created (Init(), enters a function), then starts executing (Execute() is called). It runs bytecode instructions until it yields, hits an error, or the function returns. If the function returns (natural completion), End() is called and the thread terminates itself (remove if terminateOnExit is true). If the thread yields, it posts a delayed event to resume later. If the thread is killed externally (KillThread or thread dying), the interpreter sets threadDying = true which breaks the execution loop.

8.4 Manual Control Threads

Actors (idAI, idPlayer, idWeapon) use a special "manual control" pattern for their script threads. These threads call ManualControl() which prevents automatic self-scheduling—instead, the C++ entity's Think() or UpdateScript() method calls thread->Execute() directly each frame. This gives

the C++ code full control over when and how often the script runs. The weapon script thread is a prime example: idWeapon::UpdateScript() calls weaponThread->Execute() in a loop up to 10 times per frame, allowing rapid state machine transitions.

8.5 Thread Management Functions

Function	Description
`sys.killthread( string name )`	Kills all threads whose names match the given pattern. Supports wildcard (*) at the end.
`sys.threadname( string name )`	Sets the name of the current thread.
`sys.terminate( float threadNum )`	Terminates a specific thread by number.

Note: *The interpreter has a runaway loop detector: if a thread executes more than 5,000,000 instructions without yielding, it triggers an error. Always include a yield point (waitFrame, wait, or similar) in infinite loops.*

9. scriptEvent Declarations

The **scriptEvent** keyword declares a function prototype that maps to a C++ event handler. These declarations tell the script compiler that a function exists and what its signature is, but the implementation lives in C++. This is the primary mechanism for exposing engine functionality to scripts.

9.1 Syntax

```
// From doom_events.script:
scriptEvent  void    remove();
scriptEvent  float   getHealth();
scriptEvent  void    setHealth( float health );
scriptEvent  void    setOrigin( vector origin );
scriptEvent  entity  getEntity( string name );
```

scriptEvent declarations can only appear at global (namespace) scope, not inside object definitions. The declared name must exactly match the name string in the corresponding idEventDef constructor on the C++ side. The parameter types and return type must also match the event's format string.

9.2 How scriptEvent Functions Are Called

When script code calls a scriptEvent function on an entity or via sys.*, the compiler emits either OP_EVENTCALL (for entity method calls) or OP_SYSCALL (for sys.* calls). At runtime, the interpreter marshals the script arguments into the correct C++ format using the event's format string and dispatches to the C++ handler via ProcessEvent(). For entity calls, the entity's class hierarchy determines which C++ handler responds (via the eventMap). For sys calls, idThread's own event map handles the dispatch.

> ***Note:*** *scriptEvent functions cannot be launched with the thread keyword. They execute synchronously within the calling thread. Some events (like wait and waitFrame) are "multi-frame events" that pause the interpreter and resume it later, but this is not the same as thread creation.*

10. Namespaces

DoomScript supports namespaces for organizing definitions:

```
namespace my_game {
    float global_difficulty;
    void setupDifficulty() {
        global_difficulty = 1.0;
    }
};

// Access from outside: my_game::setupDifficulty();
```

Namespaces function similarly to C++ namespaces—they provide a scoping mechanism for variable and function names. The default namespace is the global def_namespace. Objects and scriptEvent declarations must be at namespace scope (not inside functions). In practice,

namespaces are rarely used in Doom 3 scripts; most code lives in the default global namespace or inside object definitions.

11. Variable Declarations & Scope

11.1 Global Variables

Variables declared at namespace scope are global and persist for the lifetime of the program:

```
float game_difficulty;
string current_level;
```

Global variables are allocated in the idProgram::variables[] byte array and initialized to zero. They can be initialized with a constant value at declaration:

```
float max_monsters = 32;
```

11.2 Local Variables

Variables declared inside a function body are local to that function and allocated on the interpreter's local stack:

```
void weapon_shotgun::Fire() {
    float ammoClip;            // local variable
    float currentTime;         // local variable
    ammoClip = ammoInClip();
    currentTime = sys.getTime();
}
```

Local variables can be initialized with expressions (not just constants). The compiler emits OP_STORE_* instructions to perform runtime initialization.

11.3 Object Fields

Variables declared inside an object {} block are fields (member variables) of that object. They are stored in the idScriptObject's data buffer and persist for the entity's lifetime:

```
object monster_base : ai {
    float       run_distance;    // stored in script object memory
    boolean     run;
    entity      current_path;
    string      customAnim;
};
```

Object fields are accessed via the dot operator from outside or directly by name from within the object's methods. Fields inherit from parent objects: all parent fields are available in derived types. The compiler tracks field offsets to ensure proper memory layout matching the inheritance chain.

11.4 Multiple Declarations

Multiple variables of the same type can be declared on one line:

```
float x, y, z;     // three float variables
```

12. Compiler & Runtime Limits

Limit	Value	Description

Max string length	`MAX_STRING_LEN = 128`	Maximum bytes per string variable
Max local stack	`LOCALSTACK_SIZE = 6144`	Bytes available for local variables per thread
Max call stack depth	`MAX_STACK_DEPTH = 64`	Maximum function call nesting depth
Runaway instruction limit	`5,000,000`	Max instructions per Execute() call before error
Max events	`MAX_EVENTS = 4096`	Maximum number of registered idEventDefs
Max event arguments	`D_EVENT_MAXARGS = 8`	Maximum parameters per event function
Max variables	`sizeof(variables)`	Total global variable space (fixed byte array in idProgram)

13. Common Patterns & Idioms

13.1 The State Machine Pattern

Weapons and AI use a state machine pattern where each state is a method that runs in a loop and transitions by calling weaponState() or setState():

```
void weapon_shotgun::Idle() {
    weaponReady();
    playCycle( ANIMCHANNEL_ALL, "idle" );
    while( 1 ) {
        if ( WEAPON_LOWERWEAPON ) {
            weaponState( "Lower", IDLE_TO_LOWER );
        }
        if ( WEAPON_ATTACK && ammoInClip() > 0 ) {
            weaponState( "Fire", IDLE_TO_FIRE );
        }
        waitFrame();
    }
}
```

weaponState() and setState() work by killing the current thread's execution and restarting it with the new state function. The second parameter is the animation blend time for the transition.

13.2 The eachFrame Macro

```
// Defined in doom_defs.script as:
#define eachFrame  for( 0; 1; waitFrame() )

// Used in AI scripts for continuous monitoring:
void follower::Legs_Idle() {
    idleAnim( ANIMCHANNEL_LEGS, "idle" );
    eachFrame {
        if ( run ) { animState( ANIMCHANNEL_LEGS, "Legs_Run", 4 ); }
        if ( AI_FORWARD ) { animState( ANIMCHANNEL_LEGS, "Legs_Walk", 4 );
}
    }
}
```

13.3 The waitUntil Macro

```
// Defined as:
#define waitUntil( x )  while( !( x ) ) { waitFrame(); }

// Common usage - wait for animation to finish:
playAnim( ANIMCHANNEL_ALL, "reload" );
waitUntil( animDone( ANIMCHANNEL_ALL, RELOAD_TO_IDLE ) );
```

13.4 Spawn Arg Access

Entities can read their spawn arguments (key-value pairs from the entityDef or map editor) using these scriptEvent functions:

```
float   value  = getFloatKey( "shotDelay" );
string  name   = getKey( "target" );
vector  offset = getVectorKey( "offset" );
entity  tgt    = getEntityKey( "target" );
float   intVal = getIntKey( "count" );
```

13.5 Signal Handling

Signals allow entities to trigger script callbacks on specific events:

```
// Register a callback for when entity is touched:
sys.onSignal( SIG_TOUCH, $trigger_door, "onTouched" );

// Later, clear the signal:
sys.clearSignalThread( SIG_TOUCH, $trigger_door );
```

Signal types include SIG_TOUCH, SIG_USE, SIG_TRIGGER, SIG_REMOVED, SIG_DAMAGE, SIG_BLOCKED, and mover position signals (SIG_MOVER_POS1 through SIG_MOVER_2TO1).

13.6 Entity Spawning from Script

```
// Set up spawn args, then spawn:
sys.setSpawnArg( "classname", "monster_demon_imp" );
sys.setSpawnArg( "origin", "100 200 0" );
sys.setSpawnArg( "angle", "90" );
entity imp = sys.spawn( "monster_demon_imp" );
```

14. Debugging DoomScript

Several tools are available for debugging scripts at runtime:

Tool / CVar	Description
`g_debugScript 1`	Enables verbose script debug output: thread creation/destruction, function entry, line numbers.
`sys.print( string )`	Print to the console (no newline).
`sys.println( string )`	Print to the console with a newline.
`sys.assert( float cond )`	Break in debug builds if condition is zero.
`sys.error( string msg )`	Trigger a script error with a message.
`sys.warning( string msg )`	Print a warning message to the console.
`listThreads`	Console command to list all active script threads with their names, states, and call stacks.

When a script error occurs, the interpreter prints the file name, line number, thread name, and a description of the error. Common runtime errors include: null entity access (calling a method on

a $reference that hasn’t spawned), stack overflow (too-deep function call nesting), runaway loop (infinite loop without yield), and type mismatch errors in event calls.

> ***Note:*** *String concatenation with + is your primary debug tool. Build debug strings like:*
> *sys.println("Health: " + getHealth() + " Ammo: " + ammoInClip());*

15. Quick Reference Card

15.1 Type Summary

```
float       x = 42;                 // numeric (int and float)
vector      v = '1 0 0';            // 3D vector
string      s = "hello";            // fixed-length string
entity      e = $player1;           // entity reference
boolean     b = true;               // true/false
void                                // function return type only
```

15.2 Object Definition

```
object myObject : parentObject {
    float       myField;
    void        init();             // constructor
    void        destroy();          // destructor
    void        myMethod();
};

void myObject::myMethod() { ... }
```

15.3 Control Flow

```
if ( cond ) { } else { }
while ( cond ) { waitFrame(); }
for ( init; cond; incr ) { }
do { } while ( cond );
break;    continue;    return value;
```

15.4 Special Syntax

```
$entityname          // named entity reference
sys.functionName()   // system function call
thread myFunc()      // launch concurrent thread
object::method()     // scope resolution for method definition
vec_x vec_y vec_z    // vector component access
#define NAME value   // preprocessor constant
```